

Лабораторная работа 5

Установим DRF

```
$ pip install djangorestframework
Collecting djangorestframework
  Downloading djangorestframework-3.15.2-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: django>=4.2 in e:\repositories\rgpu_7sem\venv\lib\site-packages (from djangorestframework) (5.1.2)
Requirement already satisfied: asgiref<4,>=3.8.1 in e:\repositories\rgpu_7sem\venv\lib\site-packages (from django>=4.2->djangorestframework) (3.8.1)
Requirement already satisfied: sqlparse>=0.3.1 in e:\repositories\rgpu_7sem\venv\lib\site-packages (from django>=4.2->djangorestframework) (0.5.1)
Requirement already satisfied: tzdata in e:\repositories\rgpu_7sem\venv\lib\site-packages (from django>=4.2->djangorestframework) (2024.2)
Downloading djangorestframework-3.15.2-py3-none-any.whl (1.1 MB)
1.1/1.1 MB 5.6 MB/s eta 0:00:00
Installing collected packages: djangorestframework
```

Сортировка и фильтрация данных

создадим сериализатор и представление для API

```
from rest_framework import serializers
from .models import Question, Choice, TextAnswer

class ChoiceSerializer(serializers.ModelSerializer):
    class Meta:
        model = Choice
        fields = ['id', 'choice_text', 'votes']

class TextAnswerSerializer(serializers.ModelSerializer):
    class Meta:
        model = TextAnswer
        fields = ['id', 'answer_text', 'created_at']

class QuestionSerializer(serializers.ModelSerializer):
    choices = ChoiceSerializer(many=True, read_only=True, source='choice_set')
    text_answers = TextAnswerSerializer(many=True, read_only=True, source='textanswer_set')
    total_votes = serializers.SerializerMethodField()

    class Meta:
        model = Question
        fields = ['id', 'question_text', 'pub_date', 'question_type', 'choices', 'text_answers', 'total_votes']

    def get_total_votes(self, obj):
        return sum(choice.votes for choice in obj.choice_set.all())
```

```

51
52 class QuestionListAPI(generics.ListAPIView):
53     serializer_class = QuestionSerializer
54     filter_backends = [filters.OrderingFilter]
55     ordering_fields = ['pub_date', 'choice__votes']
56
57     def get_queryset(self):
58         queryset = Question.objects.annotate(
59             total_votes=Count('choice__votes'),
60             popularity=Count('choice__votes') / (
61                 timezone.now() - F('pub_date')
62             ).days
63         ).all()
64
65         # Фильтрация по date
66         start_date = self.request.query_params.get('start_date', None)
67         end_date = self.request.query_params.get('end_date', None)
68         if start_date:
69             queryset = queryset.filter(pub_date__gte=start_date)
70         if end_date:
71             queryset = queryset.filter(pub_date__lte=end_date)
72         # Сортировка
73         sort_by = self.request.query_params.get('sort_by', None)
74         if sort_by == 'date':
75             queryset = queryset.order_by('-pub_date')
76         elif sort_by == 'popularity':
77             queryset = queryset.order_by('-popularity')
78         elif sort_by == 'votes':
79             queryset = queryset.order_by('-total_votes')
80
81         return queryset
82
83
84 class QuestionDetailAPI(generics.RetrieveAPIView):
85     queryset = Question.objects.all()
86     serializer_class = QuestionSerializer
87

```

Теперь у вас есть API endpoints для сортировки и фильтрации голосований: GET /polls/api/questions/ - список всех голосований с возможностью сортировки и фильтрации:

Параметры сортировки (sort_by):

date - по дате публикации

popularity - по популярности

votes - по количеству голосов

Параметры фильтрации:

start_date - начальная дата (формат: YYYY-MM-DD)

end_date - конечная дата (формат: YYYY-MM-DD)

GET /api/questions/<id>/ - детальная информация о конкретном голосовании

Примеры использования API:Сортировка по дате: /api/questions/?sort_by=date

Сортировка по голосам: /polls/api/questions/?sort_by=votes

Фильтрация по дате:/questions/?start_date=2024-01-01&end_date=2024-12-31

API возвращает данные в формате JSON, включая:Текст вопроса

Дату публикации

Тип вопроса

Варианты ответов с количеством голосов

Общее количество голосов

Текстовые ответы (если есть)

Question List Api

GET /api/questions/

HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 5,
      "question_text": "Что такое многопроцессорность?",
      "pub_date": "2025-01-04T16:45:36.025833+03:00",
      "question_type": "text",
      "choices": [],
      "text_answers": [
        {
          "id": 2,
          "answer_text": "Многопроцессорность (multiprocessing) в Python - это способ параллельного выполнения кода с использованием...",
          "created_at": "2025-01-04T16:45:45.110957+03:00"
        }
      ],
      "total_votes": 0
    }
  ]
}
```

Django REST framework nestessia8

[Question List Api](#)

Question List Api

[Filters](#) [OPTIONS](#) [GET](#)

GET /api/questions/?start_date=2024-01-01&end_date=2024-12-31

HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "count": 0,
  "next": null,
  "previous": null,
  "results": []
}
```

Django REST framework nestessia8

[Question List Api](#) / [Question Detail Api](#)

Question Detail Api

[OPTIONS](#) [GET](#)

GET /api/questions/5/

HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "id": 5,
  "question_text": "Что такое multiprocessing?",
  "pub_date": "2025-01-04T16:45:36.025833+03:00",
  "question_type": "text",
  "choices": [],
  "text_answers": [
    {
      "id": 2,
      "answer_text": "Многопроцессорность (multiprocessing) в Python - это способ параллельного выполнения кода с использованием нескольких процессоров.",
      "created_at": "2025-01-04T16:45:45.110957+03:00"
    }
  ],
  "total_votes": 0
}
```

Экспорт данных

views.py

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from django.http import HttpResponse
from polls.models import Question

import csv
import logging
import json

logger = logging.getLogger(__name__)

class ExportDataAPI(APIView):
    def get(self, request, format=None, *args, **kwargs):
        try:
            logger.debug(f"Request path: {request.path}")
            logger.debug(f"Request GET params: {request.GET}")

            # Определяем формат из пути URL или из параметра запроса
            if request.path.endswith('.csv'):
                export_format = 'csv'
            elif request.path.endswith('.json'):
                export_format = 'json'
            else:
                export_format = request.query_params.get('format',
'json').lower()
```

```
        if export_format not in ['json', 'csv']:

            return Response(

                {'error': f'Unsupported format: {export_format}.
Use json or csv.'},

                status=status.HTTP_400_BAD_REQUEST

            )
```

```
        logger.debug(f"Requested format: {export_format}")
```

```
        questions = Question.objects.all()
```

```
        # Подготовим данные
```

```
        data = []
```

```
        for question in questions:
```

```
            question_data = {
```

```
                'id': question.id,
```

```
                'question_text': question.question_text,
```

```
                'pub_date': question.pub_date.strftime(
```

```
                    '%Y-%m-%d %H:%M:%S'
```

```
                ),
```

```
                'question_type': question.question_type,
```

```
            }
```

```
        if question.question_type == 'text':
```

```
            question_data['answers'] = [
```

```
                {
```

```
                    'text': answer.answer_text,
```

```
                    'created_at': answer.created_at.strftime(
```

```
                        '%Y-%m-%d %H:%M:%S'
```

```

        )

    }

    for answer in question.textanswer_set.all()

]

else:

    question_data['choices'] = [

        {

            'text': choice.choice_text,

            'votes': choice.votes

        }

        for choice in question.choice_set.all()

    ]

    question_data['total_votes'] = sum(

        choice.votes for choice in
question.choice_set.all()

    )

    data.append(question_data)

# В зависимости от формата возвращаем разный response
if export_format == 'csv':

    response = HttpResponse(

        content_type='text/csv',

        headers={

            'Content-Disposition': 'attachment;
filename="polls_data.csv"'

        },

    )

```

```

writer = csv.writer(response)

writer.writerow([

    'Question ID',

    'Question Text',

    'Publication Date',

    'Question Type',

    'Total Votes',

    'Choices/Answers'

])

for item in data:

    if item['question_type'] == 'text':

        answers = '; '.join(

            [a['text'] for a in item.get('answers',

[[]])

            ]

        )

    else:

        answers = '; '.join(

            [f"{c['text']} ({c['votes']} votes)"

            for c in item.get('choices', [])]

            )

    writer.writerow([

        item['id'],

        item['question_text'],

        item['pub_date'],

        item['question_type'],

        item.get('total_votes', 0),

        answers

```



```
    ]))

    return response

else: # json

    response = HttpResponse(

        content_type='application/json',

        headers={

            'Content-Disposition': 'attachment;

filename="polls_data.json"'

        },

    )

    json.dump(data, response, indent=2)

    return response

except Exception as e:

    logger.error(f"General error: {str(e)}")

    return Response(

        {'error': str(e)},

        status=status.HTTP_500_INTERNAL_SERVER_ERROR

    )
```

обновим основной urls.py и urls.py приложения

```
ls_export 9+, M | views.py ...\lr5\mysite\polls | urls.py ...\polls | urls.py ...\lr5\mysite\mysite |
Программирование > lr5 > mysite > polls_export > urls.py > ...
1  from django.urls import path
2  from . import views
3
4  app_name = 'polls_export'
5  urlpatterns = [
6      path('', views.ExportDataAPI.as_view(), name='export-data'),
7      path('.json', views.ExportDataAPI.as_view(), name='export-data-json'),
8      path('.csv', views.ExportDataAPI.as_view(), name='export-data-csv'),
9  ]
10 | Ctrl+L to chat, Ctrl+K to generate
```

```
_export 9+, M | views.py ...\lr5\mysite\polls | urls.py ...\polls | urls.py ...\lr5\mysite\mysite |
Программирование > lr5 > mysite > mysite > urls.py > ...
1  from django.contrib import admin
2  from django.urls import path, include
3
4  urlpatterns = [
5      path('admin/', admin.site.urls),
6      path('polls/', include('polls.urls')),
7      path('api/', include([
8          path('polls/', include('polls_api.urls')),
9          path('export/', include('polls_export.urls')),
10     ])),
11 ]
```

Теперь можно использовать новый API следующим образом: Для экспорта в

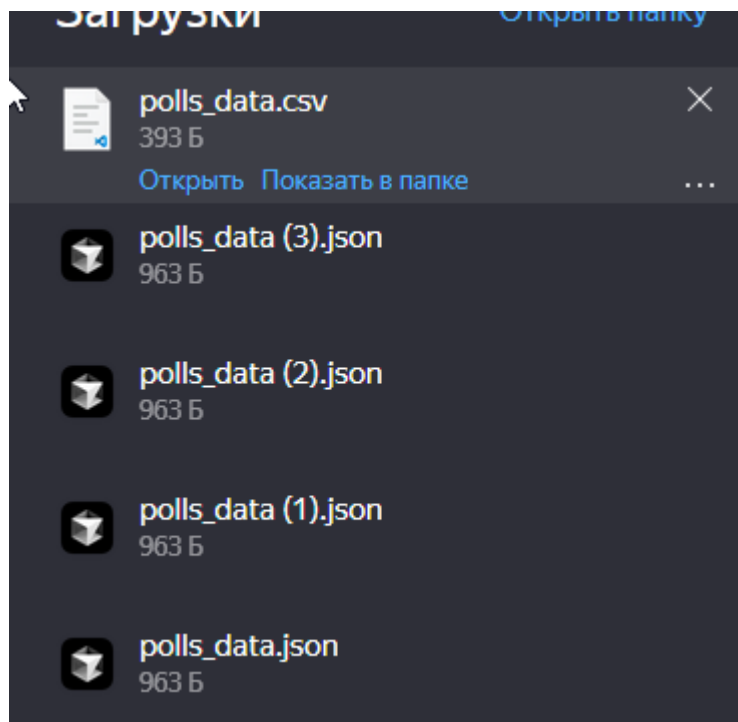
/api/export/.json

/api/export/.csv

/api/export/?format=json

/api/export/?format=csv

Теперь по этим эндпоинтам будут скачиваться файлы



Функционал страницы для поиска голосований

Создаем шаблон для страницы поиска

```
{% extends 'base.html' %}

{% block content %}
<div class="container mt-4">
  <div class="row">
    <!-- Форма поиска -->
    <div class="col-12 mb-4">
      <div class="card">
        <div class="card-body">
          <h5 class="card-title">Поиск голосований</h5>
          <form id="searchForm" class="row g-3">
            <div class="col-md-4">
              <label for="dateFrom"
class="form-label">Дата от</label>
              <input type="date" class="form-control"
id="dateFrom">
            </div>
            <div class="col-md-4">
              <label for="dateTo" class="form-label">Дата
до</label>
              <input type="date" class="form-control"
id="dateTo">
            </div>
            <div class="col-md-4">
              <label for="questionType"
class="form-label">Тип вопроса</label>
              <select class="form-select"
id="questionType">
                <option value="">Все</option>
                <option value="single">С одним
вариантом</option>
                <option value="multiple">С несколькими
вариантами</option>
                <option value="text">Текстовый
ответ</option>
              </select>
            </div>
            <div class="col-12">
              <button type="submit" class="btn
btn-primary">Поиск</button>
            </div>
          </form>
        </div>
      </div>
    </div>
  </div>
</div>
{% endblock %}
```

```

        <button type="button" class="btn
btn-success ms-2" id="exportBtn">
            Экспортировать результаты
        </button>
    </div>
</form>
</div>
</div>
</div>

<!-- Результаты поиска -->
<div class="col-md-6">
    <div class="card">
        <div class="card-body">
            <h5 class="card-title">Найденные голосования</h5>
            <div id="questionsList" class="list-group">
                <!-- Здесь будут отображаться найденные
голосования -->
            </div>
        </div>
    </div>
</div>

<!-- Детали голосования -->
<div class="col-md-6">
    <div class="card">
        <div class="card-body">
            <h5 class="card-title">Детали голосования</h5>
            <div id="questionDetails">
                <!-- Здесь будут отображаться детали выбранного
голосования -->
            </div>
        </div>
    </div>
</div>
</div>

<script>
document.addEventListener('DOMContentLoaded', function() {
    const searchForm = document.getElementById('searchForm');
    const questionsList = document.getElementById('questionsList');
    const questionDetails = document.getElementById('questionDetails');

```

```
const exportBtn = document.getElementById('exportBtn');

// Обработка отправки формы поиска
searchForm.addEventListener('submit', async function(e) {
    e.preventDefault();

    const dateFrom = document.getElementById('dateFrom').value;
    const dateTo = document.getElementById('dateTo').value;
    const questionType =
document.getElementById('questionType').value;

    // Формируем параметры запроса
    const params = new URLSearchParams();
    if (dateFrom) params.append('date_from', dateFrom);
    if (dateTo) params.append('date_to', dateTo);
    if (questionType) params.append('question_type', questionType);

    try {
        const response = await
fetch(`/api/polls/?${params.toString()}`);
        const data = await response.json();

        // Очищаем список
        questionsList.innerHTML = '';

        // Отображаем результаты
        data.forEach(question => {
            const item = document.createElement('a');
            item.href = '#';
            item.className = 'list-group-item
list-group-item-action';
            item.textContent = question.question_text;
            item.dataset.questionId = question.id;

            item.addEventListener('click', (e) => {
                e.preventDefault();
                loadQuestionDetails(question.id);
            });

            questionsList.appendChild(item);
        });
    } catch (error) {
        console.error('Error:', error);
    }
});
```

```

    }
  });

  // Загрузка деталей голосования
  async function loadQuestionDetails(questionId) {
    try {
      const response = await fetch(`/api/polls/${questionId}/`);
      const question = await response.json();

      let detailsHtml = `
        <h4>${question.question_text}</h4>
        <p>Дата публикации: ${new
Date(question.pub_date).toLocaleString()}</p>
        <p>Тип:
${getQuestionTypeText(question.question_type)}</p>
      `;

      if (question.question_type === 'text') {
        detailsHtml += '<h5>ОТВЕТЫ:</h5><ul>';
        question.answers.forEach(answer => {
          detailsHtml += `<li>${answer.text}</li>`;
        });
        detailsHtml += '</ul>';
      } else {
        detailsHtml += '<h5>Варианты ответов:</h5>';
        const totalVotes = question.choices.reduce((sum,
choice) => sum + choice.votes, 0);

        question.choices.forEach(choice => {
          const percentage = totalVotes ? ((choice.votes /
totalVotes) * 100).toFixed(1) : 0;
          detailsHtml += `
            <div class="mb-2">
              <div>${choice.text} (${choice.votes}
голосов, ${percentage}%</div>
              <div class="progress">
                <div class="progress-bar"
role="progressbar"
                style="width: ${percentage}%"
                aria-valuenow="${percentage}"
                aria-valuemin="0"
                aria-valuemax="100"></div>
            </div>
          `;
        });
      }
    }
  }

```

```

        </div>

        `;

    });

}

    questionDetails.innerHTML = detailsHtml;
} catch (error) {
    console.error('Error:', error);
}
}

// Обработка экспорта
exportBtn.addEventListener('click', function() {
    const format = window.confirm('Выберите формат:\nOK -
JSON\nCancel - CSV') ? 'json' : 'csv';
    window.location.href = `/api/export/.${format}`;
});

// Вспомогательная функция для получения текста типа вопроса
function getQuestionTypeText(type) {
    const types = {
        'single': 'С одним вариантом',
        'multiple': 'С несколькими вариантами',
        'text': 'Текстовый ответ'
    };
    return types[type] || type;
}
});
</script>
{% endblock %}

```


Обновляем urls.py

```
from django.urls import path

from . import views

app_name = 'polls'
urlpatterns = [
    path('', views.IndexView.as_view(), name='index'),
    path('<int:pk>/', views.DetailView.as_view(), name='detail'),
    path('<int:pk>/results/', views.ResultsView.as_view(), name='results'),
    path('create/', views.create_poll, name='create_poll'),
    path('<int:question_id>/vote/', views.vote, name='vote'),
    path('search/', views.search_view, name='search'),
]
```

И обновляем view для поддержки микросервисов

```
from django.db.models import F
from django.http import HttpResponseRedirect
from django.shortcuts import get_object_or_404, render
from django.urls import reverse
from django.views import generic
from django.utils import timezone
from django.shortcuts import redirect
from django.forms import formset_factory
from django.contrib.auth.decorators import login_required

from .forms import QuestionForm, ChoiceForm, ChoiceFormSet
from .models import Choice, Question, TextAnswer

class IndexView(generic.ListView):
    template_name = "polls/index.html"
    context_object_name = "latest_question_list"

    def get_queryset(self):
        """
        Return all published questions (not including those set to be
        published in the future), ordered by publication date.
        """
        return Question.objects.filter(
            pub_date__lte=timezone.now()
        ).order_by("-pub_date")
```

```

class DetailView(generic.DetailView):
    model = Question
    template_name = "polls/detail.html"

class ResultsView(generic.DetailView):
    model = Question
    template_name = "polls/results.html"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        if self.object.question_type == 'text':
            context['text_answers'] =
self.object.textanswer_set.all().order_by('-created_at')
        return context

def vote(request, question_id):
    question = get_object_or_404(Question, pk=question_id)

    if question.question_type == 'text':
        text_answer = request.POST.get("text_answer", "").strip()
        if not text_answer:
            return render(
                request,
                "polls/detail.html",
                {
                    "question": question,
                    "error_message": "Необходимо ввести ответ.",
                },
            )
        # Сохраняем текстовый ответ
        TextAnswer.objects.create(
            question=question,
            answer_text=text_answer
        )
        return HttpResponseRedirect(reverse("polls:results",
args=(question.id,)))

    elif question.question_type == 'multiple':
        # Получаем список выбранных вариантов
        selected_choices = request.POST.getlist("choice")

```

```

        if not selected_choices:
            return render(
                request,
                "polls/detail.html",
                {
                    "question": question,
                    "error_message": "Выберите хотя бы один вариант
ответа.",
                },
            )
        try:
            for choice_id in selected_choices:
                selected_choice = question.choice_set.get(pk=choice_id)
                selected_choice.votes = F("votes") + 1
                selected_choice.save()
        except (KeyError, Choice.DoesNotExist):
            return render(
                request,
                "polls/detail.html",
                {
                    "question": question,
                    "error_message": "Выбран недопустимый вариант
ответа.",
                },
            )
    else: # single choice
        try:
            selected_choice =
question.choice_set.get(pk=request.POST["choice"])
        except (KeyError, Choice.DoesNotExist):
            return render(
                request,
                "polls/detail.html",
                {
                    "question": question,
                    "error_message": "Вы не выбрали вариант ответа.",
                },
            )
    else:
        selected_choice.votes = F("votes") + 1
        selected_choice.save()

```

```

        return HttpResponseRedirect(reverse("polls:results",
args=(question.id,)))

@login_required
def create_poll(request):
    choice_formset_class = formset_factory(ChoiceForm,
formset=ChoiceFormSet, extra=2, min_num=2)

    if request.method == 'POST':
        question_form = QuestionForm(request.POST)
        choice_formset = choice_formset_class(request.POST)

        is_valid_formset = True
        if question_form.data.get('question_type') != 'text':
            is_valid_formset = choice_formset.is_valid()

        if question_form.is_valid() and is_valid_formset:
            # Сохраняем вопрос
            question = question_form.save(commit=False)
            question.pub_date = timezone.now()
            question.save()

            # Сохраняем варианты ответа только для не текстовых
вопросов
            if question.question_type != 'text':
                for choice_form in choice_formset:
                    if choice_form.cleaned_data.get('choice_text'):
                        choice = choice_form.save(commit=False)
                        choice.question = question
                        choice.save()

            return redirect('polls:index')
        else:
            question_form = QuestionForm()
            choice_formset = choice_formset_class()

    return render(request, 'polls/create_poll.html', {
        'question_form': question_form,
        'choice_formset': choice_formset
    })

```

```
def search_view(request):  
    return render(request, 'polls/search.html')
```

Результат:

[Опросы](#) [Главная](#) [Создать опрос](#) nestessia8 [Выйти](#)

Поиск голосований

Дата от
03.01.2025

Дата до
11.01.2025

Тип вопроса
Текстовый ответ

[Поиск](#) [Экспортировать результаты](#)

Найденные голосования

Что такое многопроцессорность?

Детали голосования

127.0.0.1:8000 Polls App

Опросы

Главная

Создать опрос

nestessia8

Выйти

Подтвердите действие на сайте 127.0.0.1

Выберите формат:
OK - JSON
Cancel - CSV

[Да](#) [Нет](#)

Поиск голосований

Дата от
03.01.2025

Дата до
11.01.2025

Тип вопроса
Текстовый ответ

[Поиск](#) [Экспортировать результаты](#)

Найденные голосования

Что такое многопроцессорность?

Детали голосования